

需求文档

地理定位游戏平台与 Wiki 系统开发文档

1. 文档概述

1.1 项目背景

本项目拟建设一个统一的网站平台，包含两个相互独立但可共用账号体系与基础设施的子系统：

1. **Wiki 文档系统**：服务于新加入实验室的同学，用于快速了解实验室科研方向、常用工具、论文阅读路径、实验规范、数据集、训练评测流程等内容。

2. **图片地理定位游戏平台**：核心业务系统，用于收集玩家在图像地理定位任务中的思维过程数据，并提供人与 AI 的对战能力。

项目整体要求使用 **React 技术栈**，采用**前后端分离架构**，界面现代、清晰、美观，并具备高可复用性与良好的扩展能力，以便未来接入更多任务模式、更多 AI 能力以及更多数据收集流程。

2. 项目目标

2.1 总体目标

构建不同入口的网站，将两部分完全分开，当作两个项目来开发，其中：

- Wiki 部分用于知识沉淀与新成员 onboarding；
- 核心应用部分用于数据标注、玩家参与、积分激励、AI 对战与后续研究数据管理；
- 两个系统在视觉风格、路由结构、信息架构上明显区分，避免用户混淆；

- 整体支持后续快速扩展新的图片模式、对战模式、标注类型、奖励机制与 AI 接口。

2.2 设计原则

- **前后端分离**：前端 React 应用与后端 API 服务解耦。
 - **模块化**：模式管理、标注流程、对战流程、用户系统、积分系统独立设计。
 - **高复用性**：统一的图片渲染组件、地图组件、计时组件、结果页组件、任务配置组件。
 - **可扩展性**：支持未来新增模式（例如音频地理定位、视频地理定位、文本描述定位）。
 - **数据优先**：所有用户行为、标注内容、对战结果都可被结构化记录，便于科研分析。
 - **研究友好**：支持导出数据、管理任务池、追踪标注质量、管理 AI 模型版本。
-

3. 系统范围

3.1 子系统划分

A. Wiki 文档系统（附属功能）

- 基于 **Sphinx** 构建；
- 使用 **Read the Docs Theme**；
- 主要承载实验室新人科研入门内容；
- 可独立部署为静态文档服务，也可通过主站路由集成。

B. 地理定位游戏平台（核心功能）

- 核心业务应用；
- 独立于 Wiki 的 UI 风格与交互逻辑；
- 提供：
 - 数据收集与标注；
 - AI 对战；
 - 用户中心；
 - 积分与奖励系统；

- 管理后台；
 - 数据导出与分析接口。
-

4. 用户角色

4.1 普通用户 / 玩家

主要职责：

- 注册登录；
- 参与标注任务；
- 参与 AI 对战；
- 查看积分、等级、历史记录；
- 兑换奖品。

4.2 标注管理员

主要职责：

- 配置数据收集任务；
- 管理图片来源；
- 审核用户标注；
- 查看标注质量与完成进度；
- 调整任务奖励。

4.3 系统管理员

主要职责：

- 管理用户、权限与封禁；
- 管理奖品库存；
- 管理 AI 模型接入配置；
- 管理模式开关；
- 查看系统日志与运行状态。

4.4 研究人员

主要职责：

- 创建数据采集实验；
 - 下载结构化标注数据；
 - 观察用户行为与 AI 对战表现；
 - 对比不同模式、不同 AI 版本的数据质量。
-

5. 总体架构设计

5.1 架构概览

建议采用如下架构：

- **前端**：React + TypeScript + Vite / Next.js（推荐 Next.js 作为统一 Web 框架）
- **后端 API**：Node.js（NestJS）或 Python（FastAPI）
- **数据库**：PostgreSQL
- **缓存 / 队列**：Redis
- **对象存储**：MinIO / S3（存储图片、全景快照、标注截图、奖品图片）
- **地图服务**：Mapbox / Google Maps / OpenStreetMap + Leaflet
- **鉴权**：JWT + Refresh Token
- **Wiki 文档**：Sphinx 静态构建产物
- **部署**：Docker + Nginx + CI/CD

5.2 推荐技术选型

前端

- React
- TypeScript
- Next.js（App Router）
- Zustand 或 Redux Toolkit（状态管理）

- TanStack Query（服务端状态管理）
- Tailwind CSS + shadcn/ui（现代 UI）
- React Hook Form + Zod（表单与校验）
- React Image/Pan Viewer 或 OpenSeadragon（图片缩放、拖拽）
- Leaflet / Mapbox GL JS（地图）
- ECharts / Recharts（数据统计）

后端

- NestJS（更适合大型模块化系统）
- Prisma ORM 或 TypeORM
- PostgreSQL
- Redis
- BullMQ（异步任务，如积分结算、导出、审核队列）
- Swagger/OpenAPI（接口文档）

文档系统

- Sphinx
- Read the Docs Theme
- MyST Parser（如需支持 Markdown）

5.3 前后端职责划分

前端职责

- 页面展示与交互；
- 图片缩放拖拽、框选标注、地图显示；
- 用户输入与提交流程控制；
- 倒计时、对战局展示；
- 用户中心与积分展示。

后端职责

- 鉴权与权限控制；
- 任务分发与模式配置；
- 标注数据持久化；

- AI 调用与结果缓存；
- 积分计算；
- 奖励兑换；
- 审核、统计与导出；
- 外部 API 接入与数据同步。

6. 信息架构与产品结构

6.1 顶层站点结构

代码块

```
1  /
2  |— /wiki           # Wiki 文档系统
3  |— /app            # 核心游戏平台入口
4  |   |— /home
5  |   |— /annotate   # 数据收集
6  |   |— /battle     # AI 对战
7  |   |— /history    # 历史记录
8  |   |— /points     # 积分中心
9  |   |— /rewards    # 奖品兑换
10 |   |— /profile    # 用户中心
11 |   |— /leaderboard # 排行榜
12 |— /auth
13 |   |— /login
14 |   |— /register
15 |   |— /forgot-password
16 |— /admin          # 管理后台
17 |   |— /users
18 |   |— /tasks
19 |   |— /images
20 |   |— /ai-models
21 |   |— /rewards
22 |   |— /reviews
23 |   |— /analytics
```

6.2 Wiki 与核心应用的视觉区分建议

Wiki 风格

- 白底或浅灰背景；
- 经典文档布局；
- 左侧目录树 + 右侧正文；
- 强调内容层级与可检索性；
- 使用 Sphinx 默认 RTD 风格。

核心应用风格

- 更现代、卡片化、强交互；
 - 深浅主题均可支持；
 - 强调图像、地图、计时、结果反馈；
 - 使用更明显的品牌色与游戏化动效；
 - 页面布局更“产品化”而非“文档化”。
-

7. Wiki 文档系统开发方案

7.1 功能定位

Wiki 不是主业务，但应稳定、易维护、可快速更新，承载以下内容：

- 实验室介绍；
- 新人入门路线；
- 常用论文与工具；
- 数据集说明；
- 实验规范；
- 训练与评测流程；
- 常见问题。

7.2 技术方案

- 使用 Sphinx 初始化项目；

- 主题使用 `sphinx_rtd_theme`；
- 内容使用 reStructuredText 或 Markdown（推荐 Markdown + MyST）；
- 构建输出静态 HTML；
- 由 Nginx 或对象存储静态托管；
- 主站中通过 `/wiki` 路由反向代理或静态挂载。

7.3 Wiki 目录建议

代码块

```
1  wiki/
2  |— index.md
3  |— getting-started/
4  |   |— lab-intro.md
5  |   |— environment-setup.md
6  |   |— reading-list.md
7  |   |— research-workflow.md
8  |— datasets/
9  |— tools/
10 |— experiments/
11 |— faq/
12 |— glossary/
```

7.4 部署方式

- 独立构建为静态资源；
- 每次文档仓库更新后自动构建；
- 发布到 `/wiki` 路径；
- 可接入全文搜索（如 Algolia DocSearch 或本地 lunr）。

8. 核心应用功能设计

8.1 数据收集模块

8.1.1 功能目标

收集用户在图片地理定位任务中的认知过程与显式标注数据，用于后续科研分析、模型训练、推理可解释性研究与人机对比分析。

8.1.2 任务模式

系统应支持多种图片模式，并允许未来扩展：

当前模式：

- 街景模式
- 遥感模式
- 地图模式
- 地形图模式
- 混合模式

未来预留：

- 夜景模式
- 室内模式
- 历史影像模式
- 视频定位模式
- 多图联合定位模式

建议将模式设计为可配置枚举，而非硬编码页面。

模式配置字段建议

- 模式 ID
- 模式名称
- 模式描述
- 支持的数据源类型
- 是否允许全景
- 是否允许单图

- 是否显示真实位置
- 是否启用某类标注方式
- 默认奖励分值
- 是否启用

8.1.3 标注模式

用户选择图片模式后，还需要选择标注模式：

A. 完整思维过程标注

用户输入：

- 解题思路 / 推理过程；
- 根据图片得出的判断结论；
- 可以额外填写置信度、候选地区、排除依据等。

建议结构化字段：

- 原始自由文本思维链
- 最终答案文本
- 置信度（1-5 或 0-100）
- 候选地点列表
- 关键证据摘要

B. 地理要素标注

用户需要：

- 在图像中框选一个或多个关键区域；
- 为每个区域填写标签；
- 解释该区域如何帮助定位。

建议支持的要素标签：

- 路牌
- 语言文字
- 建筑风格
- 地形地貌
- 植被

- 道路标线
- 车辆/车牌
- 电线杆/基础设施
- 商铺招牌
- 气候痕迹
- 太阳方位/阴影
- 其他

每个标注框记录：

- bbox 坐标
- 标签类别
- 文本说明
- 对定位的贡献度（可选）

8.1.4 图片展示与交互

系统需要支持两类图像来源：

图像来源

1. 外部 API 接入的全景图

- 例如 Google Street View、Mapillary 或其他街景/全景服务；
- 可能需要根据经纬度、朝向、视角动态生成可视化画面。

2. 服务器内部数据集图片

- 本地存储或对象存储中的单图、多图、遥感图、地图截图等。

前端交互要求

- 支持放大、缩小；
- 支持拖拽浏览；
- 支持全屏查看；
- 支持标注框绘制；
- 支持多框管理；
- 支持在不同分辨率下保持标注坐标一致；
- 支持全景图与静态图统一封装。

核心组件建议

- `ImageViewer`：统一图片查看器
- `PanoramaViewer`：全景图查看器
- `AnnotationCanvas`：标注层
- `BBoxEditor`：框选编辑器
- `LocationMap`：真实位置地图组件

8.1.5 真实位置展示

用户在标注时可看到图片真实对应的地理位置。这里应支持配置：

- 对某些实验任务显示真实位置；
- 对某些任务延迟显示或仅标注后显示；
- 对 AI 对战模式则通常不显示。

建议展示形式：

- 小地图 + Marker；
- 经纬度文本；
- 行政区层级信息（国家/州/省/城市）。

8.1.6 标注流程

建议完整流程如下：

1. 用户进入数据收集主页；
2. 选择图片模式；
3. 选择标注模式；
4. 系统分配一个任务样本；
5. 用户查看图片与位置；
6. 用户完成输入或框选；
7. 前端校验内容完整性；
8. 提交到后端；
9. 后端记录标注、发放积分；
10. 前端展示奖励反馈，并进入下一题或返回主页。

8.1.7 标注质量控制

为了保证数据质量，建议加入以下机制：

- 最低填写字数要求；
- 框选数量限制与下限；
- 随机插入金标准样本；
- 重复样本一致性检查；
- 管理员抽样审核；
- 恶意标注检测；
- 每用户质量分。

8.1.8 数据结构设计

标注任务表 AnnotationTask

- id
- image_id
- mode_type
- annotation_type
- status
- reward_points
- source_type
- difficulty
- created_at
- updated_at

标注记录表 AnnotationRecord

- id
- user_id
- task_id
- image_id
- mode_type
- annotation_type
- thought_text

- final_answer_text
- confidence_score
- submitted_at
- quality_status
- reward_points_granted

标注框表 AnnotationBBox

- id
- annotation_record_id
- x
- y
- width
- height
- label_type
- explanation
- contribution_score

图像资源表 ImageAsset

- id
 - source_type
 - storage_url
 - external_ref
 - latitude
 - longitude
 - country
 - region
 - city
 - image_meta_json
 - mode_tags
 - created_at
-

8.2 AI 对战模块

8.2.1 功能目标

允许用户与平台接入的 AI 模型进行地理定位挑战，以游戏化方式提升平台活跃度，并收集人类与 AI 的对比行为数据。

8.2.2 对战模式

应参考 GeoGuessr 和图寻，但做成平台内可配置模式。

图片形式

- 全景模式
- 单图模式
- 多图模式（未来）

时间规则

- 不限时
- 10 秒 / 30 秒 / 60 秒 / 120 秒
- 自定义倒计时模板

轮数规则

- 1 轮
- 3 轮
- 5 轮
- 10 轮

模式选择项

- 数据源模式（街景、遥感等）
- 图片呈现方式（全景 / 单图）
- 时间限制
- AI 对手
- 轮数

- 是否允许查看历史猜测轨迹（未来）

8.2.3 AI 对手接入

系统应支持多个 AI 模型版本接入，且能便于替换和扩展。

AI 接口抽象层设计

设计统一 AI 适配接口：

- 输入：图像资源、模式、时间限制、回合上下文
- 输出：
 - 猜测经纬度
 - 猜测文本说明（可选）
 - 模型置信度（可选）
 - 推理摘要（可选）
 - 返回时间
 - 模型版本

建议后端模块

- `AiBattleService`
- `AiProviderAdapter`
- `AiResultCacheService`
- `AiModelRegistry`

8.2.4 对战流程

1. 用户选择对战配置；
2. 系统生成本局比赛；
3. 每轮分配图片样本；
4. 用户在规定时间内提交猜测地点；
5. AI 同时返回自己的猜测；
6. 后端计算用户与 AI 分数；
7. 进入下一轮；
8. 全部结束后展示总分、胜负、逐轮对比与总结页面。

8.2.5 结果展示

建议结果页包含：

- 用户总分
- AI 总分
- 胜负结果
- 每轮对比表
- 用户猜测点与真实点距离
- AI 猜测点与真实点距离
- 回放/复盘入口
- 分享结果卡片（未来）

8.2.6 评分规则建议

可参考 GeoGuessr 逻辑，基于真实点与猜测点的距离给分。

示例：

- 距离越近分数越高；
- 每轮满分 5000；
- 最终累加。

也可支持不同模式下不同评分曲线：

- 街景模式：细粒度评分；
- 遥感模式：允许更大误差半径；
- 地形图模式：可按行政区域命中加分。

8.2.7 AI 对战数据记录

BattleSession

- id
- user_id
- ai_model_id
- battle_mode
- time_limit_sec

- round_count
- status
- started_at
- finished_at
- winner_type
- user_total_score
- ai_total_score

BattleRound

- id
 - session_id
 - round_index
 - image_id
 - ground_truth_lat
 - ground_truth_lng
 - user_guess_lat
 - user_guess_lng
 - ai_guess_lat
 - ai_guess_lng
 - user_distance_km
 - ai_distance_km
 - user_score
 - ai_score
 - round_winner_type
 - elapsed_ms_user
 - elapsed_ms_ai
-

8.3 用户系统

8.3.1 注册登录

必须提供完整的用户账户能力：

- 邮箱注册
- 用户名登录
- 邮箱验证码 / 密码登录
- 忘记密码
- 登录态保持
- 第三方登录（未来，如 Google / GitHub）

8.3.2 用户资料

包括：

- 用户名
- 头像
- 邮箱
- 积分
- 等级
- 历史标注次数
- 对战胜率
- 勋章（未来）

8.3.3 权限模型

建议 RBAC：

- user
 - reviewer
 - researcher
 - admin
-

8.4 积分与奖励系统

8.4.1 积分获取规则

用户可通过以下方式获取积分：

- 完成完整思维过程标注；
- 完成地理要素标注；
- 高质量标注额外奖励；
- 每日签到（可选）；
- 对战参与奖励（可选）；
- 特殊活动奖励（未来）。

8.4.2 积分发放策略

建议基础分 + 质量浮动分：

- 基础完成分；
- 审核通过额外分；
- 金标准命中额外分；
- 恶意行为扣分。

8.4.3 奖品兑换

用户可在奖励中心使用积分兑换奖品。

奖品信息包括：

- 奖品名称
- 所需积分
- 库存
- 图片
- 描述
- 状态（上架/下架）

兑换流程：

1. 用户选择奖品；
2. 校验积分和库存；
3. 扣减积分；
4. 生成兑换记录；

5. 管理员后续发放。

奖励相关表

RewardItem

- id
- name
- description
- points_cost
- stock
- cover_image
- status

RewardRedemption

- id
 - user_id
 - reward_item_id
 - points_spent
 - status
 - created_at
 - fulfilled_at
-

8.5 历史记录与排行榜

历史记录

用户可查看：

- 过往标注记录；
- 过往 AI 对战记录；
- 已获得积分明细；
- 奖品兑换记录。

排行榜

建议提供：

- 总积分排行榜；
 - 周榜 / 月榜；
 - AI 对战胜率榜；
 - 标注数量榜；
 - 高质量标注榜（未来）。
-

8.6 管理后台

功能范围

管理后台建议单独作为 Admin 应用或主应用中的管理员路由。

管理模块

- 用户管理
- 图片资源管理
- 标注任务管理
- 标注审核
- AI 模型管理
- 积分规则管理
- 奖品管理
- 统计分析
- 系统配置

图片资源管理

支持：

- 导入本地图像数据；
- 配置外部 API 图像引用；

- 编辑元信息（经纬度、国家、城市、难度、模式标签）；
- 数据清洗与去重。

标注审核

支持：

- 查看标注文本；
- 查看框选结果；
- 审核通过/拒绝；
- 质量标签打分；
- 批量处理。

AI 模型管理

支持：

- AI 模型名称、版本、接口地址、状态；
 - 开关启用；
 - 限流配置；
 - 返回字段校验。
-

9. 前端设计方案

9.1 前端项目结构建议

代码块

```
1  src/
2  |— app/
3  |   |— (public)/
4  |   |— (auth)/
5  |   |— (main)/
6  |   └─ admin/
7  |— components/
8  |   |— common/
9  |   └─ layout/
```

```
10 | | └─ image/
11 | | └─ map/
12 | | └─ annotate/
13 | | └─ battle/
14 | | └─ points/
15 | └─ features/
16 | | └─ auth/
17 | | └─ user/
18 | | └─ annotation/
19 | | └─ battle/
20 | | └─ rewards/
21 | | └─ leaderboard/
22 | | └─ admin/
23 | └─ services/
24 | └─ hooks/
25 | └─ store/
26 | └─ lib/
27 | └─ types/
28 | └─ styles/
```

9.2 核心前端组件

通用组件

- Button
- Modal
- Drawer
- Tabs
- Card
- Pagination
- EmptyState
- LoadingState
- ErrorState

业务组件

- ModeSelector
- AnnotationTypeSelector

- ImageViewer
- PanoramaViewer
- AnnotationToolbar
- BBoxLayer
- ThoughtInputPanel
- MapAnswerPanel
- CountdownTimer
- BattleScoreBoard
- RoundSummaryCard
- RewardCard
- PointsBadge

9.3 页面清单

用户侧页面

- 首页
- 登录页
- 注册页
- 数据收集模式选择页
- 标注任务页
- AI 对战配置页
- AI 对战进行页
- AI 对战结果页
- 用户中心
- 积分中心
- 奖品中心
- 历史记录页
- 排行榜页

管理端页面

- Dashboard
- 用户管理页

- 图片管理页
- 任务配置页
- 标注审核页
- AI 模型配置页
- 奖品管理页
- 数据统计页

9.4 UI/UX 设计要求

- 核心应用与 Wiki 视觉显著区分；
 - 以深色导航 + 明亮内容区或高对比品牌色为主；
 - 图像展示页面避免杂乱；
 - 标注流程聚焦、低打扰；
 - 对战流程强调节奏感与反馈；
 - 积分与奖励页面具备游戏化氛围；
 - 移动端支持基础浏览，桌面端优先保证完整体验。
-

10. 后端设计方案

10.1 后端模块划分

建议 NestJS 模块：

- AuthModule
- UserModule
- ImageAssetModule
- AnnotationModule
- BattleModule
- AiModule
- RewardModule
- PointsModule

- ReviewModule
- LeaderboardModule
- AdminModule
- AnalyticsModule
- ConfigModule

10.2 API 风格

- RESTful API 为主；
- 复杂统计可单独增加聚合接口；
- 文件上传使用 multipart；
- 后端输出统一响应结构；
- 使用 OpenAPI 自动生成接口文档。

10.3 示例接口设计

鉴权

- `POST /api/auth/register`
- `POST /api/auth/login`
- `POST /api/auth/refresh`
- `POST /api/auth/logout`
- `GET /api/auth/me`

模式与任务

- `GET /api/modes`
- `GET /api/annotation/tasks/next`
- `POST /api/annotation/records`
- `GET /api/annotation/history`

AI 对战

- `POST /api/battles`
- `GET /api/battles/:id`
- `POST /api/battles/:id/rounds/:roundId/submit`
- `GET /api/battles/:id/result`

积分与奖品

- GET /api/points/me
- GET /api/points/history
- GET /api/rewards
- POST /api/rewards/redeem

管理端

- POST /api/admin/images/import
- GET /api/admin/annotations/review
- POST /api/admin/annotations/:id/approve
- POST /api/admin/annotations/:id/reject
- POST /api/admin/ai-models

10.4 异步任务

建议异步处理以下场景：

- 批量导入图片；
 - 调用慢速 AI；
 - 标注质量分析；
 - 排行榜刷新；
 - 导出数据；
 - 奖品兑换通知。
-

11. 数据库设计建议

11.1 核心实体总览

- User
- Role
- Mode
- ImageAsset
- AnnotationTask

- AnnotationRecord
- AnnotationBBox
- BattleSession
- BattleRound
- AiModel
- PointsLedger
- RewardItem
- RewardRedemption
- ReviewRecord
- SystemConfig

11.2 用户相关表

User

- id
- username
- email
- password_hash
- avatar_url
- role
- points_balance
- level
- status
- created_at
- updated_at

PointsLedger

- id
- user_id
- change_amount
- balance_after
- reason_type

- reference_id
- created_at

11.3 AI 模型表

AiModel

- id
- name
- version
- provider_type
- endpoint
- auth_config_json
- timeout_ms
- enabled
- created_at
- updated_at

11.4 审核表

ReviewRecord

- id
 - annotation_record_id
 - reviewer_id
 - review_status
 - review_score
 - comments
 - created_at
-

12. 外部 API 接入设计

12.1 全景图 API 接入需求

如接入外部全景服务，需抽象统一接口，避免业务层直接依赖某一家供应商。

12.2 统一数据适配层

建议设计：

- PanoramaProviderAdapter
- ImageSourceResolver
- GeoMetadataNormalizer

12.3 外部 API 风险点

- 配额限制；
- 响应延迟；
- 服务稳定性；
- 授权密钥泄漏；
- 返回数据格式不统一。

12.4 缓存策略

- 静态元数据可缓存；
 - 热门样本可预取；
 - AI 结果可按样本 + 模型版本缓存；
 - 对外部图像 URL 进行签名控制。
-

13. 可扩展性设计

13.1 模式插件化

新增模式时，理想情况下只需：

- 在模式配置中新增一条记录；
- 补充对应前端渲染逻辑或复用现有逻辑；
- 新增必要的评分规则与任务分发策略。

13.2 AI 任务扩展

未来不仅支持“地理定位对战”，还可扩展为：

- AI 与人类进行地理要素识别对战；
- AI 与人类进行思维链解释对战；
- 多模型排行榜；
- AI 协作标注模式；
- 人类纠错 AI 模式。

13.3 数据采集扩展

未来可扩展采集：

- 鼠标轨迹
- 放大缩小行为日志
- 停留时长
- 标注顺序
- 键盘输入过程
- 语音思维过程

为此建议预留行为事件表：

- session_id
 - user_id
 - event_type
 - event_payload_json
 - created_at
-

14. 安全设计

14.1 基础安全

- JWT + Refresh Token；
- 密码加密存储；
- 接口鉴权与权限校验；
- 限流；

- CSRF / XSS / SQL 注入防护；
- 上传文件类型校验；
- 管理后台权限隔离。

14.2 业务安全

- 防刷积分；
- 防重复兑换；
- 任务领取与提交幂等；
- AI 对战结果防篡改；
- 审核日志留痕；
- 敏感配置加密存储。

14.3 隐私与合规

- 用户输入数据应明确告知用途；
 - 标注数据可匿名导出；
 - 外部 API 使用需符合授权协议；
 - 保留用户协议与隐私政策页面。
-

15. 性能与可用性设计

15.1 前端性能

- 图片分级加载；
- 全景资源懒加载；
- 组件按需加载；
- 路由级代码拆分；
- 长列表虚拟滚动。

15.2 后端性能

- 高频接口缓存；
- 数据库索引优化；

- 批量写入；
- 异步队列处理重任务；
- 热点排行榜预计算。

15.3 可用性

- 关键接口重试；
 - AI 服务降级；
 - 外部 API 故障时切换内部数据；
 - 图片加载失败兜底；
 - 审核与导出任务可恢复。
-

16. 埋点与科研数据支持

16.1 必须记录的数据

- 用户 ID
- 任务 ID
- 模式
- 图片 ID
- 开始时间 / 提交时间
- 完成耗时
- 提交内容
- 标注框详情
- AI 结果
- 对战胜负

16.2 建议记录的行为数据

- 图片缩放次数
- 平均缩放倍率
- 拖拽路径
- 首次输入时间

- 修改次数
- 对战倒计时下的提交时机

16.3 数据导出能力

管理后台应支持：

- 按模式导出；
 - 按时间范围导出；
 - 按用户群体导出；
 - 按 AI 模型版本导出；
 - JSON / CSV 格式导出。
-

17. 测试方案

17.1 前端测试

- 单元测试：组件、hooks、工具函数；
- 集成测试：标注提交流程、对战流程；
- E2E 测试：登录、标注、兑换、对战。

工具建议：

- Vitest
- Testing Library
- Playwright

17.2 后端测试

- 单元测试：服务层、积分计算、评分逻辑；
- 集成测试：数据库与 API；
- 合约测试：AI 适配器接口。

工具建议：

- Jest
- Supertest

17.3 测试重点

- 图片标注坐标准确性；
 - 倒计时与提交边界；
 - 幂等性；
 - 积分一致性；
 - 高并发下奖品库存；
 - 外部 API 失败恢复。
-

18. 部署方案

18.1 部署架构

建议容器化部署：

- frontend-web
- backend-api
- postgres
- redis
- minio
- nginx
- wiki-static

18.2 环境划分

- dev
- staging
- production

18.3 CI/CD

建议流程：

- Push 触发测试；
- 通过后构建镜像；

- 自动部署到测试环境；
- 人工确认后部署生产。

18.4 监控

- 前端异常监控（Sentry）
 - 后端日志聚合
 - API 延迟监控
 - AI 服务调用监控
 - 数据库与缓存监控
-

19. 推荐开发里程碑

阶段 1：基础设施与账号系统

- 前后端项目初始化
- 登录注册
- 用户中心基础版
- Wiki 系统上线

阶段 2：数据收集 MVP

- 模式选择
- 标注模式选择
- 图片查看器
- 完整思维过程标注提交
- 地理要素框选提交
- 积分发放基础版

阶段 3：AI 对战 MVP

- 对战配置
- 单轮与多轮流程
- AI 接口接入

- 分数计算
- 结果页

阶段 4：管理后台与审核

- 图片资源导入
- 标注审核
- AI 模型管理
- 奖品管理
- 数据导出

阶段 5：优化与扩展

- 行为埋点
- 排行榜
- 奖章体系
- 性能优化
- 更多模式接入

20. 代码仓库规划建议

建议拆分为 monorepo：

代码块

```
1  repo/
2  |— apps/
3  |   |— web/           # React/Next.js 主站
4  |   |— api/           # NestJS 后端
5  |   |— admin/         # 可并入 web，也可独立
6  |   └─ wiki/          # Sphinx 文档
7  |— packages/
8  |   |— ui/             # 通用 UI 组件
9  |   |— config/         # eslint/tsconfig/tailwind 配置
10 |   |— types/          # 共享类型
11 |   |— utils/          # 公共方法
12 |   └─ sdk/            # 前端 API SDK
```

```
13  └─ infra/
14    └─ docker/
15    └─ nginx/
16    └─ scripts/
```

优点：

- 便于共享组件与类型；
 - 便于未来增加新应用；
 - 管理一致的工程规范。
-

21. 关键实现建议总结

21.1 Wiki 部分

- 用 Sphinx 快速搭建；
- 作为附属文档系统，不应占用核心应用过多开发成本；
- 静态部署即可。

21.2 核心应用部分

必须围绕三件事设计：

1. 高质量采集用户地理推理数据；
2. 提供强交互、强反馈的 AI 对战体验；
3. 为未来更多研究任务与产品功能扩展预留空间。

21.3 最重要的工程原则

- 统一模式抽象；
- 统一图片展示组件；
- 统一 AI 适配层；
- 统一积分账本；
- 统一事件埋点；

- 统一管理后台。
-

22. 建议优先级

P0（必须优先实现）

- 登录注册
- 用户系统
- 模式选择
- 两类标注流程
- 图片缩放拖拽与框选
- 真实位置展示
- 积分系统基础版
- AI 对战基础版
- 对战结果页

P1（第二阶段）

- 奖品兑换
- 历史记录
- 排行榜
- 管理后台图片导入
- 标注审核
- AI 模型配置

P2（后续增强）

- 行为埋点
- 金标准样本
- 勋章系统
- 分享卡片
- 更多模式
- 多模型对战

23. 附录：MVP 范围建议

为了尽快上线，建议 MVP 只做：

Wiki

- 首页
- 新人入门
- 常用工具
- FAQ

核心应用

- 用户注册登录
- 街景 / 单图 两种模式
- 完整思维过程标注
- 地理要素标注
- 图片放大缩小拖拽
- 真实位置地图显示
- 积分累积
- 基础奖品页
- AI 单图对战
- 3 轮结果页
- 管理员上传内部图片

这样可以最快验证：

- 用户是否愿意参与；
 - 数据是否有价值；
 - AI 对战是否具备吸引力；
 - 产品结构是否适合继续扩展。
-

24. 结语

本项目虽然表面上是一个网站，但本质上应被设计为一个**面向科研的数据采集与人机对战平台**。因此，开发时不能只关注页面是否能用，更要关注：

- 数据结构是否能支撑研究；
- 模块是否便于后续扩展；
- AI 接口是否可插拔；
- 标注质量是否可控；
- 用户激励机制是否闭环。

如果按本文档推进，推荐先完成一个 **MVP 版本**，尽快上线验证真实用户参与情况，再逐步增加复杂模式、排行榜、审核、行为埋点与更多 AI 玩法。